

Botão de Merge no GitHub + Histórico Linear (Squash) — Revisada

Objetivo

Padronizar o fluxo de merge para **Squash and merge**, evitando merges que quebrem a linearidade quando `required_linear_history=true`. Isso simplifica a didática: **1 PR → 1 commit** no histórico da `main`.

O que estava errado no guia anterior (e por quê)

- **Flags inválidas no** `gh repo edit`: `--disable-merge-commit` / `--disable-rebase-merge` **não existem**. O `gh repo edit` usa apenas flags do tipo `--enable-<estilo>=true|false`.
- **Placeholder literal** `:owner/:repo`: se usado **literalmente**, retorna **404**. Ou substitua por `owner/repo` ou rode **dentro do diretório do repositório**.

Resultado: o comando falhou até trocarmos para a forma correta (`--enable-merge-commit=false`, etc.) e executarmos **dentro** do repositório.

O que funcionou e por quê

- **Executar dentro do repositório**: `gh repo edit` infere `owner/repo` do **remote** atual.
 - **Booleans explícitos**: para **desativar** um método de merge, use `--enable-<estilo>=false`; para **ativar**, `=true`.
 - **Experiência simples p/ alunos**: deixar **apenas Squash** ativo evita confusão e combina com `required_linear_history=true`.
-

Passo a passo pela UI

1. Acesse **Settings → General → Merge button** do repositório.
2. **Desmarque**: *Allow merge commits*.
3. **Marque**: *Allow squash merging*.
4. (Opcional) **Desmarque**: *Allow rebase merging* (se quiser simplificar para a turma).
5. (Opcional) Defina **Default merge style = Squash**.
6. (Opcional) **Automatically delete head branches** após o merge (mantém o repositório organizado).

Com `required_linear_history=true`, merges que criariam *merge commits* já são bloqueados. Ajustar o botão remove a opção indevida da UI e evita tentativa frustrada.

Automação por CLI (Revisada)

Opção A — Dentro do diretório do repo (funcionou)

```
gh repo edit
  --enable-merge-commit=false
  --enable-rebase-merge=false
  --enable-squash-merge=true
  --delete-branch-on-merge
```

Opção B — Informando o repositório explicitamente

```
gh repo edit owner/repo
  --enable-merge-commit=false
  --enable-rebase-merge=false
  --enable-squash-merge=true
  --delete-branch-on-merge
```

Opção C — Via API (PATCH)

```
gh api -X PATCH repos/owner/repo
-F allow_merge_commit=false
-F allow_rebase_merge=false
-F allow_squash_merge=true
-F delete_branch_on_merge=true
```

Verificação rápida

```
gh api repos/owner/repo | jq '{
  allow_merge_commit: .allow_merge_commit,
  allow_squash_merge: .allow_squash_merge,
  allow_rebase_merge: .allow_rebase_merge,
  delete_branch_on_merge: .delete_branch_on_merge
}'
```

Esperado - `allow_merge_commit: false` - `allow_squash_merge: true` - `allow_rebase_merge: false` (opcional; pode ser `true` se preferir rebase) - `delete_branch_on_merge: true` (opcional)

Por que adotar Squash com histórico linear?

- **Didática e UX:** reduz escolhas e erros dos alunos (sem “Merge commit” acidental).

- **Auditoria e rollback:** cada PR vira um único commit coeso na `main`.
 - **Consistência entre times:** mesmo padrão para todos os microserviços do monorepo.
-

Checklist para a aula

- [] `required_linear_history=true` na proteção da `main`.
- [] *Allow merge commits* **desativado**.
- [] *Allow squash merging* **ativado** (preferencial).
- [] (Opcional) *Allow rebase merging* **desativado**.
- [] (Opcional) **Auto-delete** da branch **ativado**.

Fluxo final: **Abrir PR** → **CI verde** → **1 aprovação** → **resolver conversas** → ***Squash and merge*** → **branch excluída**.